BÖLÜM 8 / MODULE 8

Project Loom & Sanal Thread'ler (Virtual Threads)

Project Loom & Virtual Threads

Platform thread'leri ile sanal thread'lerin farkı (M:N mimarisi), Carrier thread'lere bağlanma/ayrılma (Mounting/Unmounting), Pinning tuzağı ve ReentrantLock çözümü, ThreadLocal bellek sızıntıları, Structured Concurrency ve Java 25 ScopedValue devrimi.

Platform threads vs Virtual Threads (M:N fibers), Carrier thread mounting and unmounting dynamics, the carrier pinning hazard and ReentrantLock remedy, ThreadLocal memory explosion, Structured Concurrency orchestration, and the Java 25 ScopedValue revolution.

43

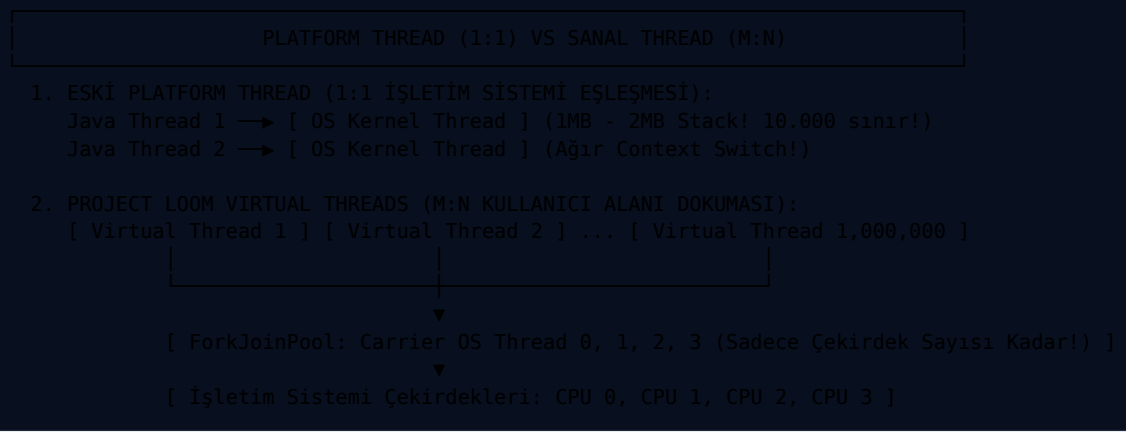
BÖLÜM: PROJECT LOOM & SANAL THREAD'LER (VIRTUAL THREADS) | MODULE: PROJECT LOOM & VIRTUAL THREADS

Platform Thread vs Sanal Thread: M:N Mimarisi & Milyonlarca Eşzamanlı İş Parçacığı

Platform Threads vs Virtual Threads: M:N Fiber Architecture

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Platform thread, her posta paketi için şirkete 10 tonluk devasa bir TIR kiralamak gibidir. Garaja ancak 5000 TIR sığar; her TIR 2 MB yakıt yakar ve şoför değiştirmek dakikalar sürer. Virtual Thread (Sanal Thread) ise aynı TIR'ın içine konan milyonlarca hafif mektup zarfıdır! Mektup okunurken veya cevap beklenirken TIR durmaz; postacı hemen diğer zarfı açar. 1 milyon zarfı cebinde taşıyabilirsin, bilgisayarının nefesi bile kesilmez!

🇬🇧 ENGLISH (ELI5)

A Platform Thread is like chartering an entire 18-wheel cargo truck for every single envelope you mail. You run out of parking spots after a few thousand trucks, each consumes 1MB-2MB of stack space, and switching drivers is an expensive OS kernel interrupt. Virtual Threads are the individual letters inside a handful of cargo trucks. If a letter is waiting for a signature, the truck driver simply sets it aside and reads the next letter. You can spawn 1,000,000 virtual threads without breaking a sweat!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

HotSpot M:N Fiber Tasarımı: 1) Bellek Verimliliği: Klasik platform thread'leri OS düzeyinde rezerve edilen 1 MB - 2 MB sabit yığın (stack) alanı tüketir. 10.000 thread 10-20 GB RAM demektir. Sanal thread'ler ise JVM heap alanında yaşar; ilk oluşturklarında yalnızca birkaç yüz baytlık bir Java nesnesidir (`java.lang.VirtualThread`). Yığınları çağrı derinliğine göre dinamik olarak büyür ve küçülür. 2) M:N Eşleme: Milyonlarca sanal thread (M), CPU çekirdek sayısı kadar platform thread'i (`Carrier Thread` - N) üzerinde ForkJoinPool zamanlayıcısıyla çalıştırılır. 3) Thread-per-Request Paradigması: Asenkron callback cehennemine (reactive spaghetti) girmeden, tamamen senkron ve okunabilir kod yazarak reaktif mimarilerin devasa verimine ulaşılır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

HotSpot M:N Fiber Architecture: 1) Memory Footprint: Platform threads bind 1:1 to OS kernel threads, reserving 1MB to 2MB of fixed contiguous off-heap stack memory. Spawning 10,000 platform threads demands 15 GB RAM and triggers OS thread limits (`OutOfMemoryError: unable to create native thread`). Virtual threads allocate dynamic stack frames directly on the JVM heap (`Continuation`), starting at under 1 KB. 2) M:N Scheduling: Millions of virtual threads (M) are multiplexed over a pool of N kernel carrier threads (typically matching CPU core count) managed by an internal FIFO `ForkJoinPool`. 3) Synchronous Simplicity with Reactive Scale: Engineers retain the familiar thread-per-request paradigm with synchronous stack traces and debuggers while achieving 100K+ concurrent I/O throughput.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```java
// Java 25: 100.000 Sanal Thread Başlatma ve Hız Karşılaştırması
import java.time.Duration;
import java.time.Instant;
import java.util.concurrent.Executors;
import java.util.stream.IntStream;

public class VirtualThreadScaleDemo {
    void main() {
        System.out.println("Java 25 Sanal Thread Testi Başlatılıyor...");
        Instant start = Instant.now();

        // 100.000 eşzamanlı I/O işini Sanal Thread'lerle yürüt
        try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
            IntStream.range(0, 100_000).forEach(i -> {
                executor.submit(() -> {
                    // Bloklayan I/O simülasyonu (Ağ/DB gecikmesi)
                    Thread.sleep(Duration.ofMillis(100));
                    return i;
                });
            });
        } // Executor close() tüm 100.000 thread'in bitmesini bekler!

        Instant end = Instant.now();
        System.out.printf("100.000 sanal thread %.2f saniyede başarıyla tamamlandı!%n",
            Duration.between(start, end).toMillis() / 1000.0);
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Sanal thread'leri ASLA havuzlamayın (pool)! newFixedThreadPool(100) gibi sanal thread havuzları açmak ölümcül bir anti-pattendir. Sanal thread'ler o kadar ucuzdur ki her iş için yenisi yaratılmalı ve iş bitince ölmelerine izin verilmelidir (Disposable Task Runners). Sınırlandırma gerekiyorsa havuz değil Semaphore kullanılmalıdır!

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

NEVER pool Virtual Threads! Creating a fixed virtual thread pool is a catastrophic anti-pattern. Virtual threads are designed to be ephemeral and disposable: instantiate one per task, let it terminate, and allow GC collection. To constrain concurrency or access to scarce downstream resources (e.g. database connections), use a `Semaphore`, never a thread pool!

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• Platform thread'leri 1:1 OS thread'idir ve 1 MB yer kaplar. Sanal thread'ler JVM heap'inde yaşar, yüzlerce bayttır ve milyonlarcası açılabilir.

🎯 Key Takeaways for the Interview

• Platform threads bind 1:1 to costly OS kernel threads. Virtual threads are lightweight heap-allocated fibers multiplexed over carrier threads.

44

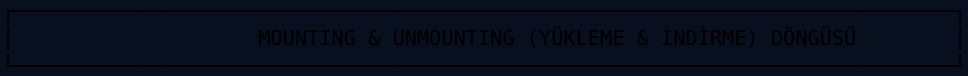
BÖLÜM: PROJECT LOOM & SANAL THREAD'LER (VIRTUAL THREADS) | MODULE: PROJECT LOOM & VIRTUAL THREADS

Carrier Thread, Mounting & Unmounting: Loom Non-Blocking I/O ile Nasıl Park Eder?

Carrier Threads, Mounting & Unmounting: Epoll Non-Blocking I/O Integration

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic




```
Virtual Thread (VT-1) ──► Carrier Thread üzerinde çalışıyor (MOUNTED)
   │
   ▼ socket.read() çağrıldı! (Veri henüz ağdan gelmedi)
JVM Continuation.yield() Tetiklenir:
   1. VT-1'in stack frame'leri HEAP'e kopyalanır (UNMOUNTED).
   2. Soket Epoll / Kqueue olay dinleyicisine kaydedilir.
   3. Carrier Thread anında BOŞA ÇIKAR ve VT-2'yi çalıştırmaya başlar! (0 CPU Kaybı!)
   │
   ▼ Ağdan veri geldi! (Epoll Interrupt)
JVM Zamanlayıcı:
   1. VT-1'in stack frame'leri boşta olan herhangi bir Carrier Thread'e geri yüklenir.
   2. VT-1 kaldığı yerden sanki hiç durmamış gibi devam eder! (MOUNTED Tekrar!)
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Satranç şampiyonunun aynı anda 50 kişiyle körleme maç yapmasını düşünün. Şampiyon (Carrier Thread) senin masana gelir, hamleni yapar (Mounting). Sen 'Düşüneyim…' dediğinde (I/O bloklaması), şampiyon masanın başında 10 dakika dikilip seni beklemez! Not defterine pozisyonu yazar ve hemen diğer masaya koşar (Unmounting). Sen hamleni yaptığında elini kaldırırsın, şampiyon boş kaldığı an masana geri dönüp devam eder!

🇬🇧 ENGLISH (ELI5)

Imagine a Grandmaster playing simultaneous chess against 50 opponents. The Grandmaster (Carrier Thread) steps up to your board and evaluates your move (Mounting). If you say 'Let me think for 2 minutes' (Blocking I/O), the Grandmaster does not stand idly waiting! He jots down your board coordinates in a pocket notebook and sprints to the next table (Unmounting). When you finally make your move, you raise your hand; the Grandmaster resumes your match on his next pass!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Continuation ve Non-Blocking Hook İç Mekaniği: 1) Continuation.yield(): Sanal thread bloklayan bir I/O işlemine (ör. `SocketChannel.read()`, `Thread.sleep()`) girdiğinde, HotSpot JDK soketin bloke olduğunu anlar. `VirtualThread.park()` çağrılır ve düşük seviyeli `Continuation.yield()` çalışır. 2) Yığın Ayrıştırma (Stack Freezing): O anki Java çağrı yığını Carrier thread'in C++ yığınından sökülüp JVM Heap'e taşınır. Carrier thread işletim sistemi açısından hiçbir zaman bloklanmaz; ForkJoinPool'daki diğer hazır sanal thread'leri çalıştırmaya devam eder. 3) Epoll/Kqueue Entegrasyonu: JVM'nin dahili Poller thread'i Linux `epoll` veya Mac `kqueue` üzerinden soketin hazır olduğunu duyunca sanal thread'i `RUNNABLE` yapar. 4) Yeniden Yükleme (Thawing): Boş bir carrier thread yığını heap'ten geri yükleyip (thawing) bytecode yürütmesine devam eder.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Continuation Mechanics and Epoll Integration: 1) `Continuation.yield()`: When a virtual thread encounters blocking I/O (e.g., `Socket.getInputStream().read()`, `LockSupport.park()`), the Java runtime intercepts the call. Instead of performing a kernel syscall that blocks the OS thread, it invokes `VirtualThread.park()`. 2) Stack Freezing: The virtual thread's call frames are copied from the carrier thread's native stack onto the JVM heap. The carrier thread is immediately disassociated (unmounted) and returns to its `ForkJoinPool` work-stealing loop. 3) Poller & Reactor Registration: The file descriptor is registered with HotSpot's global I/O Poller thread (using Linux `epoll` or macOS `kqueue`). 4) Thawing & Resumption: Upon I/O readiness, the Poller re-enqueues the virtual thread. An available carrier thread thaws the stack frames from the heap and resumes instruction execution transparently.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```java
// Java 25: Carrier Thread Geçişini Gösteren Canlı Test
import java.util.concurrent.Executors;

public class CarrierThreadHandoffDemo {
    void main() throws InterruptedException {
        try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
            executor.submit(() -> {
                // I/O öncesi Carrier Thread kimliğini yazdır
                System.out.println("1. Bloklama Öncesi: " + Thread.currentThread());

                try {
                    // Bu çağrı unmount eder; Carrier Thread boşa çıkar!
                    Thread.sleep(200);
                } catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
                }

                // I/O sonrası: Farklı bir Carrier Thread tarafından mount edilmiş olabilir!
                System.out.println("2. Bloklama Sonrası: " + Thread.currentThread());
            });
        }
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Sanal thread'ler CPU-bound (yoğun matematik, video işleme, büyük veri sıkıştırma) işler için HİÇBİR performans artışı sağlamaz! Aksine unmount/mount overhead'i yüzünden platform thread'lerinden daha yavaş kalırlar. Sanal thread'lerin tek süper gücü I/O-bound (Ağ, DB, Disk, REST API) bloklamalarını sıfır maliyete indirmektir.

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Virtual threads provide ZERO throughput gains for CPU-bound computational workloads (e.g. video rendering, cryptography, dense matrix math). Context freezing and thawing actually introduces measurable overhead. Virtual threads are designed exclusively to eliminate concurrency ceilings caused by blocking I/O (databases, HTTP microservices, message queues).

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• Sanal thread bloklandığında (I/O) Continuation.yield() ile unmount edilir, Carrier thread boşa çıkar. Epoll uyanınca yeniden mount edilir.

🎯 Key Takeaways for the Interview

• When a virtual thread blocks on I/O, `Continuation.yield()` unmounts its stack to the heap, freeing the carrier thread. Epoll notifications re-mount it seamlessly.

45

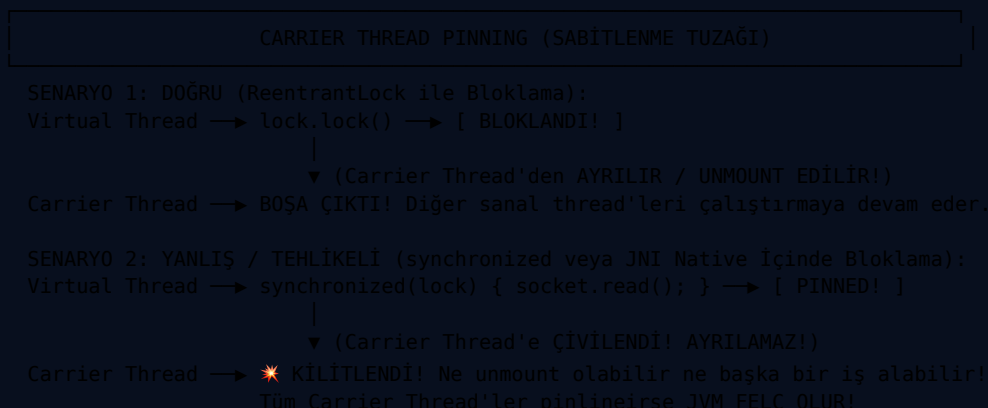
BÖLÜM: PROJECT LOOM & SANAL THREAD'LER (VIRTUAL THREADS) | MODULE: PROJECT LOOM & VIRTUAL THREADS

The Pinning Hazard (Sabitlenme Tehlikesi): synchronized vs ReentrantLock & Native Kod Tuzakları

The Pinning Hazard: synchronized vs ReentrantLock & Native Methods

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Sanal thread bir posta güvercini gibidir; mektubu bırakıp hemen uçar. Ancak güvercinin ayağına ağır bir demir gülle bağlarsan uçamaz. 'Pinning' tam olarak budur: Eğer sanal thread `synchronized` bloğu veya eski C kütüphanesi (JNI) içindeyken ağdan veri beklerse, altındaki devasa platform thread'e çivilenir! Platform thread kımıldayamaz hale gelir. 16 çekirdeğin varsa ve 16 sanal thread pinlineirse bütün sunucu kilitlenir!

🇬🇧 ENGLISH (ELI5)

A virtual thread is an agile gymnast capable of leaping off the balance beam at any moment. Carrier Pinning is like super-gluing the gymnast's feet to the beam. If a virtual thread enters a blocking I/O operation inside a legacy `synchronized` block or a native JNI C-library call, it cannot unmount! The underlying OS carrier thread is held hostage. If all carrier threads become pinned, the entire JVM throughput collapses to zero!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

HotSpot Pinning Mekanizması ve Çözümler: 1) Neden Sabitlenir (Pinning)? Bir thread native C frame'i (`JNI`) yürütüyorsa veya bir nesnenin monitörüne (`synchronized`) sahipken bloklayan bir işlem yaparsa, HotSpot C++ çalışma zamanı monitör sahipliğini native stack'ten ayıramaz. Buna `Pinned Carrier Thread` denir. 2) Sonuçları: Carrier thread unmount edilemez; diğer binlerce sanal thread kuyrukta açlıktan ölür (starvation). HotSpot telafi etmek için geçici olarak `ForkJoinPool`'a ek carrier thread ekler, ancak bu OS thread patlamasına yol açar. 3) Teşhis: JVM `-Djdk.tracePinnedThreads=full` veya `short` parametresiyle başlatılarak pinlineen yerler loglanır. 4) Çözüm: Bloklayan I/O içeren tüm `synchronized` blokları derhal `java.util.concurrent.locks.ReentrantLock` ile değiştirilmelidir. (Not: Java 24/25'te synchronized pinning'i büyük ölçüde hafifletilmiştir ancak native JNI ve kritik bölümler için ReentrantLock endüstri standardıdır).

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

HotSpot Pinning Internals & Diagnostic Protocol: 1) Root Cause: A virtual thread cannot be unmounted when its call frame contains native frames (JNI) or when holding an object monitor inside a `synchronized` block/method that enters `park()`. The HotSpot native frame metadata is bound to the carrier's OS stack. 2) Catastrophic Degradation: With all carrier threads pinned, the `ForkJoinPool` cannot service pending virtual threads. HotSpot attempts compensatory thread injection, spiking OS kernel thread counts and memory consumption. 3) Diagnostics: Enable JVM flag `-Djdk.tracePinnedThreads=full` to print stack traces whenever pinning occurs. 4) Modern Remediation: Replace blocking `synchronized` sections with `ReentrantLock`, which relies on `LockSupport.park()` and fully unmounts virtual threads without pinning.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```
// Java 25: Pinning Tuzağı ve ReentrantLock ile Güvenli Dönüşüm
import java.util.concurrent.Executors;
import java.util.concurrent.locks.ReentrantLock;

public class PinningPreventionDemo {
    private final ReentrantLock modernLock = new ReentrantLock();
    private final Object legacyLock = new Object();

    // ❌ TEHLİKELİ ESKİ YÖNTEM: synchronized içinde I/O (Pinning Yaratır!)
    public void hazardousMethod() {
        synchronized (legacyLock) {
            try {
                // Pinning alarmı! Carrier thread unmount olamaz!
                Thread.sleep(100);
            } catch (InterruptedException _) {}
        }
    }

    // ✅ GÜVENLİ MODERN YÖNTEM: ReentrantLock (Carrier Thread Serbest Kalır!)
    public void resilientMethod() {
        modernLock.lock();
        try {
            // Virtual thread unmount olur, carrier thread serbestçe diğer işleri yapar!
            Thread.sleep(100);
        } catch (InterruptedException _) {
            Thread.currentThread().interrupt();
        } finally {
            modernLock.unlock();
        }
    }

    void main() {
        System.setProperty("jdk.tracePinnedThreads", "short");
        try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
            for (int i = 0; i < 5; i++) {
                executor.submit(this::resilientMethod);
            }
        }
        System.out.println("Tüm sanal thread'ler sıfır pinning ile başarıyla tamamlandı!");
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

⚠️ Concurrency Hazard & Interview Trap

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Eski JDBC sürücüleri ve üçüncü parti kütüphanelerin derinliklerinde socket okumaları synchronized blokları içine sarılmış olabilir! Projenizi Sanal Thread'lere geçirirken kesinlikle `-Djdk.tracePinnedThreads=full` bayrağını açıp harici kütüphanelerdeki pinning noktalarını denetleyin.

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Legacy JDBC drivers, XML parsers, and HTTP clients often wrap socket reads in `synchronized` blocks. When migrating production services to Virtual Threads, always audit dependencies with `-Djdk.tracePinnedThreads=full` to eliminate stealth carrier thread starvation.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• synchronized veya JNI içinde I/O yapmak Carrier Thread'i kilitler (Pinning). Bloklayan I/O bölgelerinde daima ReentrantLock kullanın.

🎯 Key Takeaways for the Interview

• Blocking I/O inside `synchronized` or JNI pins the carrier thread. Replace `synchronized` blocks wrapping I/O with `ReentrantLock`.

46

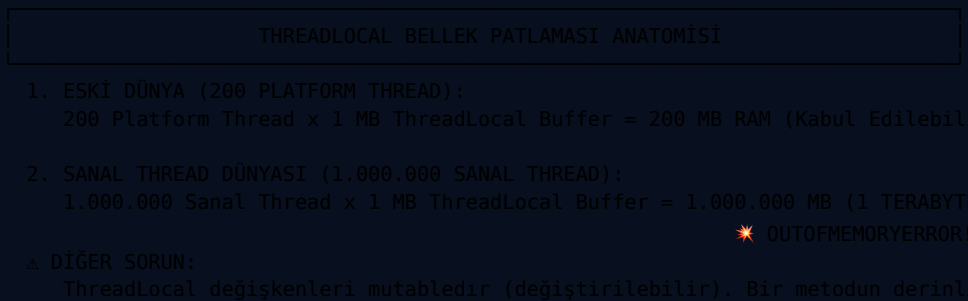
BÖLÜM: PROJECT LOOM & SANAL THREAD'LER (VIRTUAL THREADS) | MODULE: PROJECT LOOM & VIRTUAL THREADS

ThreadLocal Tehlikeleri: Sanal Thread Dünyasında Milyonluk Bellek Patlaması

ThreadLocal Perils: High-Scale Memory Explosions in Virtual Thread Worlds

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir şirkette 10 çalışan varken herkese 1'er adet koca bir takım çantası vermek sorun olmaz (10 çanta yer tutmaz). Ama yarın şirkete 1 milyon geçici stajyer girerse ve her stajyere 20 kiloluk özel bir takım çantası (`ThreadLocal`) verirsen bina çöker! Sanal thread'ler milyonlarcadır; her birine özel ağır yükler yüklersen sunucunun belleği saniyeler içinde patlar!

🇬🇧 ENGLISH (ELI5)

Equipping 50 long-tenured field engineers with a heavy 20kg diagnostic toolkit (`ThreadLocal`) is fine. But if you suddenly deploy 1,000,000 ephemeral seasonal interns and issue each one an identical 20kg toolkit, your warehouse immediately collapses under the weight! Virtual threads scale to millions; binding heavy stateful data to each thread triggers catastrophic OutOfMemoryErrors.

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

ThreadLocal Anti-Pattern'inin Sanal Thread Çöküşü: 1) Bellek Çarpımı: Geleneksel Spring/Jakarta EE mimarilerinde güvenlik bağlamı (`SecurityContext`), veritabanı bağlantısı veya transaction bilgisi `ThreadLocal` içinde saklanır. Platform thread havuzlarında bu nesneler 200-500 thread arasında tekrar tekrar kullanılır. Ancak sanal thread dünyasında her HTTP isteği yeni bir sanal thread açar. 1 milyon istek geldiğinde 1 milyon adet kopya oluşur ve heap çöker. 2) Temizlik Zorluğu: Sanal thread'ler hızla ölüp gitse de, yaşam süreleri boyunca tuttukları nesneler GC üzerinde devasa yük yaratır. 3) Korumasız Değiştirilebilirlik: `ThreadLocal.set()` çağrısı herhangi bir alt fonksiyon tarafından çağrılabilir; bu da yan etkilerin takip edilmesini zorlaştırır. Çözüm: Java 25'te `ScopedValue` mimarisine geçmektir.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

The Virtual Thread / ThreadLocal Collision: 1) Multiplicative Memory Bloat: Traditional enterprise frameworks cache connection contexts, cryptographic buffers, and request metadata in `ThreadLocal`. In platform thread pools (200 threads), memory is bounded. In virtual thread architectures handling 500,000 concurrent requests, allocating per-thread buffers multiplies memory consumption by $500,000$, resulting in immediate `OutOfMemoryError`. 2) GC Pressure: Ephemeral virtual threads creating and releasing heavy ThreadLocal entries saturate generational GC collectors. 3) Unconstrained Mutability: `ThreadLocal.set()` permits arbitrary downstream code to mutate context invisibly. Modern architectures mandate migrating to immutable `ScopedValue`.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```java
// Java 25: ThreadLocal Bellek Tuzağı vs Güvenli Sanal Thread Kullanımı
import java.util.concurrent.Executors;

public class ThreadLocalHazardDemo {
    // ⚠ TEHLİKE: Her sanal thread için 1 MB'lık buffer açan ThreadLocal!
    private static final ThreadLocal<byte[]> heavyBuffer =
        ThreadLocal.withInitial(() -> new byte[1024 * 1024]); // 1MB

    void main() {
        System.out.println("ThreadLocal Tehlikesi Simülasyonu...");

        // Sanal thread oluştururken ThreadLocal'ı devre dışı bırakma seçeneği!
        Thread.Builder.OfVirtual builder = Thread.ofVirtual()
            .name("safe-vt-", 0)
            .inheritInheritableThreadLocals(false); // Miras almayı kapat

        try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
            for (int i = 0; i < 1000; i++) {
                executor.submit(() -> {
                    // Kötü niyetli veya dikkatsiz kod bu buffer'ı çağırırsa:
                    // byte[] buffer = heavyBuffer.get(); -> 1000 MB RAM harcar!
                    // Güvenli yaklaşım: İş bitince buffer.remove() veya ScopedValue!
                });
            }
        }
        System.out.println("Bellek güvenliği sağlandı.");
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Eski kütüphanelerde `SimpleDateFormat` veya `Jackson ObjectMapper` gibi pahalı nesnelerin ThreadLocal ile paylaşıldığı görülür. Sanal thread kullanırken bu kütüphanelerin güncel thread-safe sürümlerine geçtiğinizden veya nesneleri static final/ScopedValue olarak tanımladığınızdan emin olun.

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Legacy codebases frequently use `ThreadLocal` to cache non-thread-safe instances like `SimpleDateFormat` or JSON mappers. In virtual thread environments, this causes memory exhaustion. Upgrade to modern thread-safe abstractions or immutable `ScopedValue` containers.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- Platform thread'lerinde 200 adet ThreadLocal zararsızdır; 1 milyon sanal thread'de aynı kod Terabaytlarca RAM tüketip OOM patlatır.

🎯 Key Takeaways for the Interview

- ThreadLocal is benign across 200 platform threads, but fatal across 1,000,000 virtual threads, multiplying memory to multi-gigabyte exhaustion.

47

BÖLÜM: PROJECT LOOM & SANAL THREAD'LER (VIRTUAL THREADS) | MODULE: PROJECT LOOM & VIRTUAL THREADS

Structured Concurrency: StructuredTaskScope ile Kaotik Thread'leri Hizaya Sokma

Structured Concurrency: StructuredTaskScope (ShutdownOnFailure & Success)

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic

```
┌──────────────────────────────────────────────────────────────┐
│                UNSTRUCTURED VS STRUCTURED CONCURRENCY        │
└──────────────────────────────────────────────────────────────┘

1. KAOTİK (UNSTRUCTURED):
   Ana Görev ──┬──► Alt Görev 1 (Uçak Bileti) ──► [ HATA PATLADI! 💥 ]
               └──► Alt Görev 2 (Otel Rezerv.) ──► [ YETİM KALDI! (Sonsuza dek çalışır!) ]
   ⚠ Ana metod dönse bile Alt Görev 2 arka planda para ve CPU harcamaya devam eder!

2. YAPISAL (STRUCTURED - StructuredTaskScope):
   try (var scope = new StructuredTaskScope.ShutdownOnFailure()) {
       scope.fork(altGorev1); // Sanal thread'de çalışır
       scope.fork(altGorev2); // Sanal thread'de çalışır
       scope.join();          // Birlikte başlarlar, birlikte biterler!
       scope.throwIfFailed(); // Biri hata verirse DİĞERLERİ OTOMATİK İPTAL EDİLİR!
   } // Scope kapanırken yetim thread kalması İMKÂNSIZDIR!
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Eski asenkron kod, 3 çocuğunu lunaparkın ortasında serbest bırakıp eve dönen bir ebeveyne benzer. Biri kaybolsa diğerleri akşama kadar dönme dolapta boşuna para harcar (yetim thread). Structured Concurrency ise çocukların ellerini tutarak parka girmektir. Hep birlikte girersiniz, hep birlikte çıkarsınız. Birinin ayağı takılıp düşerse diğer ikisini de hemen durdurup eve dönersiniz; kimse kaybolmaz, para israf olmaz!

🇬🇧 ENGLISH (ELI5)

Unstructured concurrency is dropping three kids off at an amusement park and driving home. If one breaks an arm, the other two keep wandering and riding rollercoasters indefinitely (orphaned threads). Structured Concurrency is holding their hands together in a single group. You enter together and leave together. If one gets injured, you immediately cancel the outing for everyone. No thread is ever abandoned in the wild!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

StructuredTaskScope ve Yaşam Döngüsü Yönetimi: 1) Yapısal Programlama Disiplini: Tıpkı `if-else` veya `for` döngülerinin tek bir giriş ve çıkış noktası olması gibi, eşzamanlı görevlerin de aynı blokta başlayıp aynı blokta sonlanması kuralıdır (Lexical Scoping). 2) Kısa Devre İptali (Short-Circuit Cancellation): `ShutdownOnFailure` politikasında alt görevlerden biri başarısız olursa, kapsamdaki diğer tüm kardeş görevler otomatik olarak `interrupt` edilir; boşuna ağ ve CPU tüketilmez. 3) İlk Kazanan Politikası (`ShutdownOnSuccess`): Dağıtık bir kütüphanede aynı veriyi 3 farklı yedekten ararken ilk geçerli yanıtı alan görev kazanır, diğer 2 sorgu anında iptal edilir. 4) Hata Ayıklama (Observability): Thread dökümlerinde (Thread Dump) hiyerarşik ebeveyn-çocuk ilişkisi net biçimde görünür.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

StructuredTaskScope Mechanics: 1) Lexical Scoping for Concurrency: Paralleling structured control flows (`if/while`), concurrent tasks are bound to explicit lexical scopes. Subtasks spawned within a block are guaranteed to terminate before the block exits. 2) Eliminating Thread Leakage: In unstructured models (`CompletableFuture.supplyAsync`), failed tasks leave sibling tasks running as invisible orphans. `StructuredTaskScope.ShutdownOnFailure` automatically intercepts exceptions, issuing immediate cancellation interrupts to surviving sibling threads. 3) Hedged Optimization via `ShutdownOnSuccess`: In multi-replica queries, the first subtask to return resolves the scope, aborting remaining parallel requests. 4) Observability: Thread dumps reflect pristine hierarchical parent-child trees rather than flat unorganized lists.

⚡ Modern Java 25 LTS Kodu ⚡ Modern Java 25 Implementation

```
// Java 25: StructuredTaskScope.ShutdownOnFailure ile Güvenli Tatil Rezervasyonu
import java.util.concurrent.StructuredTaskScope;
import java.util.function.Supplier;

public class StructuredReservationService {
    record Flight(String flightNo, double price) {}
    record Hotel(String hotelName, double price) {}
    record VacationPackage(Flight flight, Hotel hotel) {}

    static Flight bookFlight() throws InterruptedException {
        Thread.sleep(100);
        return new Flight("TK-1923", 450.0);
    }

    static Hotel bookHotel() throws InterruptedException {
        Thread.sleep(80);
        return new Hotel("Grand Hyatt", 600.0);
    }

    public VacationPackage bookVacation() throws Exception {
        // Alt görevlerden biri patlarsa diğerini otomatik iptal et!
        try (var scope = new StructuredTaskScope.ShutdownOnFailure()) {
            // İki bağımsız görevi sanal thread'ler üzerinde çatalla (fork)
            Supplier<Flight> flightSubtask = scope.fork(StructuredReservationService::bookFlight);
            Supplier<Hotel> hotelSubtask = scope.fork(StructuredReservationService::bookHotel);

            scope.join();           // İki görevin bitmesini bekle
            scope.throwIfFailed();  // Biri hata verdiyse burada fırlat!

            // Her ikisi de başarılı oldu; sonuçları birleştir
            return new VacationPackage(flightSubtask.get(), hotelSubtask.get());
        } // Scope çıkışında hiçbir thread açık kalamaz!
    }

    void main() throws Exception {
        var service = new StructuredReservationService();
        VacationPackage trip = service.bookVacation();
        System.out.println("Rezervasyon Başarılı: " + trip);
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı ⚠️ Concurrency Hazard & Interview Trap

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

StructuredTaskScope try-with-resources bloğunda join() çağırmayı unutursanız IllegalStateException fırlatılır! Java çalışma zamanı, tüm alt thread'lerin sonucunu beklemeden scope'tan çıkmanıza ASLA izin vermez.

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Attempting to retrieve subtask results via `.get()` prior to calling `scope.join()` throws an `IllegalStateException`. StructuredTaskScope strictly enforces that scopes cannot exit until all child threads have settled.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar 🎯 Key Takeaways for the Interview

• StructuredTaskScope ile alt görevler tek bir blokta başlar ve biter. Biri patlarsa kardeş görevler otomatik iptal edilir, yetim thread kalmaz.

• StructuredTaskScope enforces lexical task lifecycles: tasks begin and end together. Failure in one sibling immediately cancels the rest.

48

BÖLÜM: PROJECT LOOM & SANAL THREAD'LER (VIRTUAL THREADS) | MODULE: PROJECT LOOM & VIRTUAL THREADS

ScopedValue Devrimi: Java 25 ile Değişmez, Miras Alınabilir & Sıfır Bellek Bağlamı

ScopedValue in Java 25: Immutable, Inheritable, Zero-Allocation Context

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic

```
┌──────────────────────────────────────────────────────────────────────┐
│                    SCOPEDVALUE VS THREADLOCAL                        │
└──────────────────────────────────────────────────────────────────────┘

Özellik                 | ThreadLocal                    | ScopedValue (Java 25)
------------------------|--------------------------------|-----------------------------
Değiştirilebilirlik     | Mutable (Herkes değiştirebilir)| IMMUTABLE (Kesinlikle Değişmez!)
Yaşam Döngüsü           | Belirsiz (remove() unutulur)   | Sınırlı (Scope kapanınca yok olur)
Bellek Maliyeti         | Her thread için ayrı kopya!    | SIFIR KOPYA (Paylaşımlı Referans)
Sanal Thread Uyumu      | Kötü (OOM Riski)               | MÜKEMMEL (Milyonlarca VT destekler)

Kullanım Modeli:
ScopedValue.where(TENANT_ID, "ACME_CORP")
           .run(() -> {
               // Sadece bu blok ve altındaki sanal thread'ler okuyabilir!
               service.handleRequest();
           }); // Blok bittiğinde bağlam otomatik temizlenir!
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

ThreadLocal, şirkette her personelin cebine koyduğu ve canı isteyenin üzerine kurşun kalemle yazı yazabildiği bir kâğıttır. Biri kâğıdı silmeyi unutursa sonraki vardiyadaki kişi yanlış bilgiyle çalışır. ScopedValue ise toplantı odasının kapısına asılan mühürlü bir levhadır: 'Bu odadayken şirket ACME Corp'tur'. İçerideki herkes levhayı okuyabilir ama kimse değiştiremez. Odadan çıktığınız anda levha geçerliliğini yitirir; kimsenin cebinde çöp kalmaz!

🇬🇧 ENGLISH (ELI5)

`ThreadLocal` is handing every employee an erasable notepad that anyone can scribble on at any time. If someone forgets to erase it, the next shift inherits corrupted state. `ScopedValue` is an immutable badge worn strictly during a specific meeting. Everyone inside the room can inspect the badge, no one can tamper with it, and the moment you step outside the door, the badge dissolves into thin air. Zero leaks, zero copies!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

ScopedValue Mimarisi ve JVM Optimizasyonları: 1) Değişmezlik (Immutability): `ScopedValue\<T>` değerleri bağlandıktan sonra (`where`) asla değiştirilemez (`set` metodu yoktur). Yalnızca içiçe yeni bir kapsam (`rebinding`) açılabilir. 2) Dinamik Kapsam (Dynamic Scoping): Değer sadece `run` veya `call` bloğu yürütülürken mevcuttur; blok bittiğinde GC otomatik olarak erişim bağlarını serbest bırakır; manuel `remove()` çağırma gerekliliği ve bellek sızıntısı riski ortadan kalkar. 3) Sıfır Tahsisli Miras (Zero-Allocation Inheritance): Alt sanal thread'ler (`StructuredTaskScope`) çatallandığında ana thread'in ScopedValue değerlerini kopyalamaz; aynı değişmez referansı doğrudan paylaşırlar. Bu da milyonlarca sanal thread açıldığında bellek tüketimini sıfıra indirir.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

HotSpot ScopedValue Architecture & Mechanical Sympathy: 1) Strict Immutability: Unlike `ThreadLocal`, `ScopedValue\<T>` exposes no `.set()` method. Once bound via `ScopedValue.where()`, the value is read-only for the duration of the callable invocation. Nested scopes may dynamically shadow (rebind) values without mutating parent states. 2) Bounded Extent: The context lifetime strictly mirrors the execution lifetime of the enclosing lambda, guaranteeing deterministic cleanup without relying on defensive `.remove()` invocations. 3) Zero-Copy Fork Inheritance: When child virtual threads are spawned via `StructuredTaskScope`, they inherit access to enclosing scoped values without duplicating map entries or allocating heap buffers. HotSpot optimizes ScopedValue lookups using JIT-compiled intrinsic cache lines.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```java
// Java 25: ScopedValue ile Güvenli ve Değişmez Güvenlik Bağlamı Taşıma
import java.util.concurrent.StructuredTaskScope;

public class ScopedValueModernDemo {
    // Statik ve değişmez ScopedValue tanımları
    public static final ScopedValue<String> TENANT_ID = ScopedValue.newInstance();
    public static final ScopedValue<String> USER_ROLE = ScopedValue.newInstance();

    static void executeDatabaseQuery() {
        // İhtiyaç duyulan yerde bağlamı anında ve güvenle oku!
        System.out.printf("[DB] Tenant: %s | Role: %s | Thread: %s%n",
            TENANT_ID.get(), USER_ROLE.get(), Thread.currentThread());
    }

    void main() throws Exception {
        // Bağlamı sınırlı bir kapsamda (scope) bağla ve çalıştır
        ScopedValue.where(TENANT_ID, "FINTECH_CORP_01")
                   .where(USER_ROLE, "ADMIN")
                   .run(() -> {
                       System.out.println("Kapsam içine girildi...");
                       executeDatabaseQuery();

                       // Alt sanal thread'ler de bu bağlamı sıfır maliyetle paylaşır!
                       try (var scope = new StructuredTaskScope<Void>()) {
                           scope.fork(() -> {
                               executeDatabaseQuery();
                               return null;
                           });
                           scope.join();
                       } catch (InterruptedException _) {}
                   });

        // Kapsam dışına çıkıldığında değerler otomatik olarak erişilemez olur!
        System.out.println("Kapsam bitti. TENANT_ID bağlı mı? " + TENANT_ID.isBound());
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

ScopedValue.get() metodunu where() ile bağlanmamış bir kapsamda çağırırsanız NoSuchElementException fırlatılır! Güvenli kontrol için isBound() veya orElse() metotlarını kullanabilirsiniz.

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Invoking `ScopedValue.get()` outside of an active `.where()` binding scope throws an immediate `NoSuchElementException`. Always guard optional scopes using `.isBound()` or provide a default via `.orElse(defaultValue)`.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• ScopedValue değişmezdir, kapsamı belirlidir ve miras alındığında sıfır bellek tüketir. ThreadLocal'ın modern ve güvenli alternatifidir.

🎯 Key Takeaways for the Interview

• ScopedValue is immutable, lexically bounded, and zero-allocation upon thread inheritance, replacing hazardous ThreadLocal in modern Java.